Educational Report: Moneyball AutoML and Ensemble Notebook

A learner-friendly walkthrough of what each Python code cell is doing and how to read the output cells.

Theme: predicting Major League Baseball team wins using engineered team-season features, automated model search, and an ensemble.

NOTEBOOK WORKFLOW AT A GLANCE

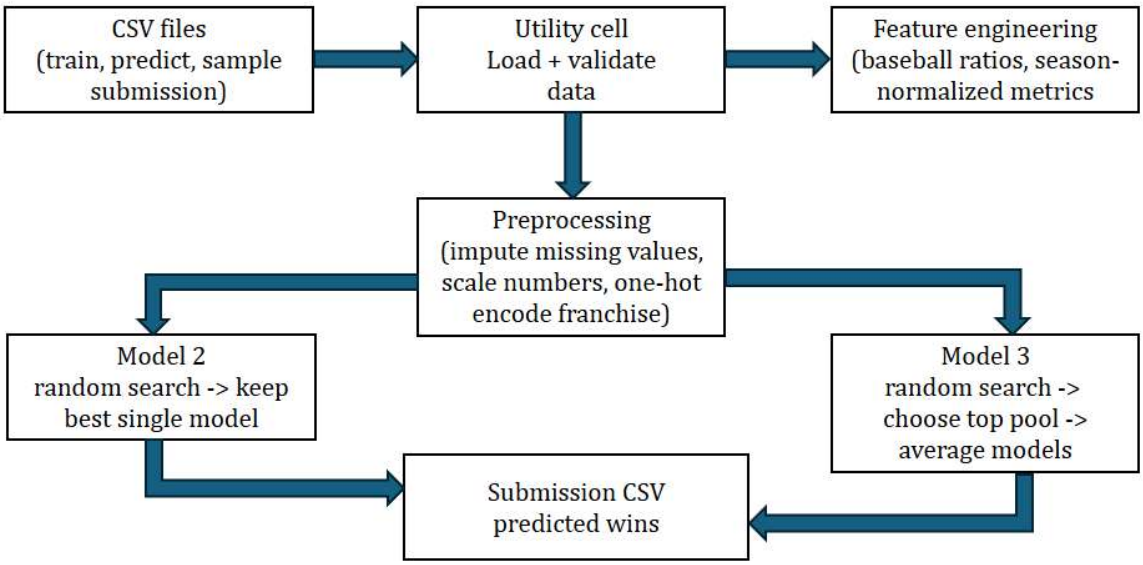


Figure 1. High-level flow of the notebook, from baseball team data to final prediction files.

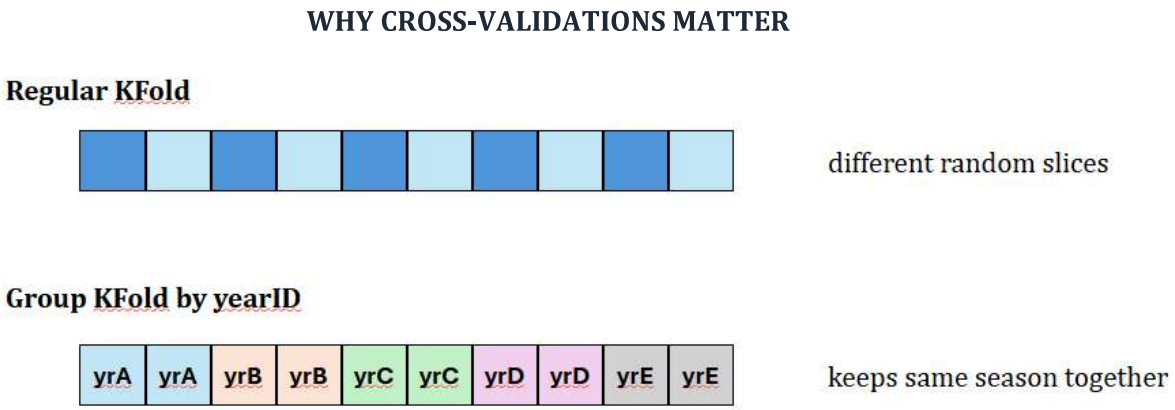
Table of Contents

Educational Report: Moneyball AutoML and Ensemble Notebook 1
How to Read This Notebook 2
Code Cell 1 - Common Utilities 3
Imports, random seeds, and file paths 4
File checking and loading 4
Safe division helper 4
Feature engineering: turning raw baseball columns into more informative signals 4
Selecting usable input columns 5
Building the preprocessing pipeline 5
Train/predict split and evaluation helper 6
Saving predictions and packaging the dataset 6
Output cell meaning 6
Code Cell 3 - Model 2 Configuration 6
Code Cell 4 - Model 2 Execution 7
Sampling candidate models 8
Preparing data and preprocessing 8
Trial loop: build pipeline, score it, and compare against the current best 8
Leaderboard, final fit, and saved submission 9
Output cell meaning 10
Code Cell 6 - Model 3 Configuration 10
Code Cell 7 - Model 3 Execution 10
Sampling and scoring candidate pipelines 11
Code Cell 9 - Output File Check 12
Key Learning Points for a New Python and Machine Learning Learner 12
Appendix A - Short Glossary 13
Appendix B - Source Note 13

How to Read This Notebook

This notebook is not training on individual players. It is training on **team-season records.** You can imagine each row as a season report card for one baseball team. The target column, **W**, is the number of wins. The model studies the relationship between baseball statistics and team wins, then predicts wins for new rows where the target is unknown.

The notebook contains two main modeling strategies. **Model 2** searches through many randomly configured machine learning models and keeps the single best one. **Model 3** searches again, keeps a pool of strong candidates, randomly picks several of them, and averages their predictions.



MAE (Mean Absolute Error) = average size of the prediction mistakes.
Example: if the model predicts 88 wins and the real value is 84, the absolute error is 4 wins. Lower MAE is better because the average miss is smaller.

Figure 2. Simple view of cross-validation and the MAE metric used throughout the notebook.

A useful metaphor is a **cooking competition.** The raw ingredients are the CSV files. Feature engineering is the chopping and seasoning. Cross-validation is the tasting panel. MAE is the average amount by which the judges say the dish missed the target taste. The final saved CSV is the plated dish sent to the competition table.

Code Cell 1 - Common Utilities

This is the foundation cell. It does not train a model yet. Instead, it prepares the tools, rules, and helper functions that every later modeling section depends on.

Imports, random seeds, and file paths

The first part imports Python libraries such as **NumPy** and **pandas** for data handling, and scikit-learn classes for preprocessing and modeling. It then fixes the random seed at **42**. A **random seed** is like fixing the shuffle order of a deck of cards so that repeated runs are reproducible. It also points Python to the project folder and names the required input files: data.csv, predict.csv, and sample_submission.csv.

File checking and loading

The function validate_input_files()checks whether the expected files actually exist. This is a protective step. Without it, later code might fail in a confusing way. The function load_base_data()then reads the CSV files into pandas DataFrames. A **DataFrame** is a spreadsheet-like table in Python.

Safe division helper

The helper _safe_divide(a, b)performs division while protecting against division by zero. In baseball data, some denominators can occasionally be zero or missing. Instead of crashing, the function returns NaN, which means "not a number" or "missing result". This is similar to leaving a blank cell in a spreadsheet rather than writing an impossible value.

Feature engineering: turning raw baseball columns into more informative signals

The function add_engineered_features()is the most important educational part of this cell. It combines the training and prediction tables temporarily so that both receive exactly the same engineered features. Then it creates new columns that summarize offense, pitching, defense, and overall team quality.

Feature group	What the code creates	Why it helps
Offense	TB, OBP_simplified, SLG, OPS, RunsCreated_basic, K_rate_approx, BB_rate_approx, HR_rate_approx, SB_rate_approx	These estimate how often a team gets on base, hits for power, strikes out, walks, steals bases, and creates runs. They convert raw counting stats into rates and quality measures.
Pitching	WHIP_calc, H_per_9, BB_per_9, SOA_per_9, HRA_per_9	These show how many baserunners or damaging events the pitching staff allows per inning or per nine innings.
Defense	FieldingPct, DP_per_game, Errors_per_game	These give clues about whether the team converts chances cleanly and avoids mistakes.
Team quality	R_per_game, RA_per_game, RunDiff, RunDiff_per_game, PythagWinPct	These directly connect scoring and run prevention to likely winning strength. Run differential is especially informative.
Era-aware normalization	For each base metric, the code adds _year_ratio and _year_z versions	Baseball environments change by era. A stat that is great in one season may be ordinary in another. These season-adjusted features help the model compare teams more fairly across years.

A helpful metaphor: raw counters are like saying the shop sold 1,000 items. Engineered features are like asking, "1,000 items out of how many visitors?" That second question is often more informative.

Selecting usable input columns

The function build_feature_lists()separates model input columns from columns that should not be used. It excludes the target **W**, identifiers such as **ID** and **teamID**, and **WinPct_proxy** because that value is derived from the target. Using target-derived information would be **data leakage.** Data leakage means the model secretly gets access to the answer while training, which makes evaluation unrealistically optimistic.

It keeps franchIDas a categorical column when present, and it keeps yearIDamong the numeric inputs because baseball scoring conditions change across eras.

Building the preprocessing pipeline

The function build_preprocessor()creates two mini-assembly lines. For numeric columns, it fills missing values with the median and then standardizes them. **Standardization** rescales values so different columns are on a more comparable scale. For categorical columns, it fills missing values with the most frequent category and then one-hot encodes them. **One-hot encoding** turns a category such as a franchise code into several yes/no columns that machine learning models can understand.

Train/predict split and evaluation helper

The function get_xy()returns X_train, y_train, and X_pred. In machine learning, **X** means inputs and **y** means the answer to learn. The function evaluate_model_cv() runs cross-validation and returns positive MAE values. The notebook uses either **KFold** or **GroupKFold**. GroupKFold keeps entire seasons together when splitting by yearID, which reduces the chance of the model learning season-specific context from both the training and validation folds at the same time.

Saving predictions and packaging the dataset

The helper write_submission() rounds predicted wins to whole numbers, clips them between 0 and 162, and writes the result into a submission CSV. The function prepare_datasets() pulls all earlier helpers together and returns a convenient dictionary of training data, prediction data, selected features, and groups.

Output cell meaning

The output says **"Utilities loaded. Ready to run any section."** This is a health check message. It tells the user that the helper functions were created successfully and no file or import error stopped execution.

Code Cell 3 - Model 2 Configuration

This cell defines the settings for the first modeling strategy. It does not run the search yet; it only writes the rules that the next cell will obey.

Setting	Value in notebook	Meaning
MODEL2_N_TRIALS	50	Try 50 randomly configured models.
MODEL2_CV_SPLITS	10	Use 10 validation folds when scoring a model.
MODEL2_USE_GROUP_CV	False	Regular KFold is used, not GroupKFold.
MODEL2_USE_GROUP_KFOLD	True	Defined, but not actually used later in the execution cell. This looks like a leftover variable.
MODEL2_GROUP_COLUMN	yearID	If group CV were enabled, seasons would be grouped by year.
MODEL2_OUTPUT_FILE	prediction_submission_2.csv	Name of the saved prediction file.

Setting	Value in notebook	Meaning
MODEL2_CANDIDATE_POOL	ridge, lasso, elasticnet, random_forest, extra_trees, gradient_boosting, hist_gradient_boosting, lightgbm	The search samples one algorithm family at a time from this pool.

For a beginner, think of this cell as setting the rules of a tournament before the games begin: how many contestants enter, how judges score them, and what the trophy file will be called.

Code Cell 4 - Model 2 Execution

This is the first full training-and-prediction cell. It performs a random search over different model families and hyperparameters, measures each trial using cross-validated MAE, keeps the best pipeline, fits it on all available training data, and saves the predictions.

Sampling candidate models

The function sample_model2_estimator(rng) randomly chooses one algorithm family from the candidate pool and then randomly draws its hyperparameters. **Hyperparameters** are settings chosen by the practitioner before training starts, such as the number of trees or the amount of regularization. It is like choosing oven temperature and baking time before baking a cake.

The notebook samples from linear models (**Ridge, Lasso, ElasticNet**) and tree-based models (**RandomForest, ExtraTrees, GradientBoosting, HistGradientBoosting, LightGBM**). This is a diverse search space: some models are simple and interpretable, while others are more flexible and nonlinear.

Preparing data and preprocessing

The cell calls prepare_datasets(), which loads the CSV files, engineers baseball features, selects usable inputs, and returns the training matrix, target, prediction matrix, and grouping column. It then builds the shared preprocessing step with missing-value imputation, scaling, and categorical encoding.

Trial loop: build pipeline, score it, and compare against the current best

For each of the 50 trials, the code creates a two-step scikit-learn **Pipeline**. Step 1 is the preprocessor. Step 2 is the sampled model. This matters because it guarantees that the same data-cleaning logic is applied consistently inside every cross-validation fold.

The code then evaluates the pipeline with MAE. Since MODEL2_USE_GROUP_CV is set to False, the cell uses regular **KFold** rather than **GroupKFold**. The average MAE and its standard deviation are recorded. If the current trial beats the best score seen so far, that pipeline becomes the new best pipeline.

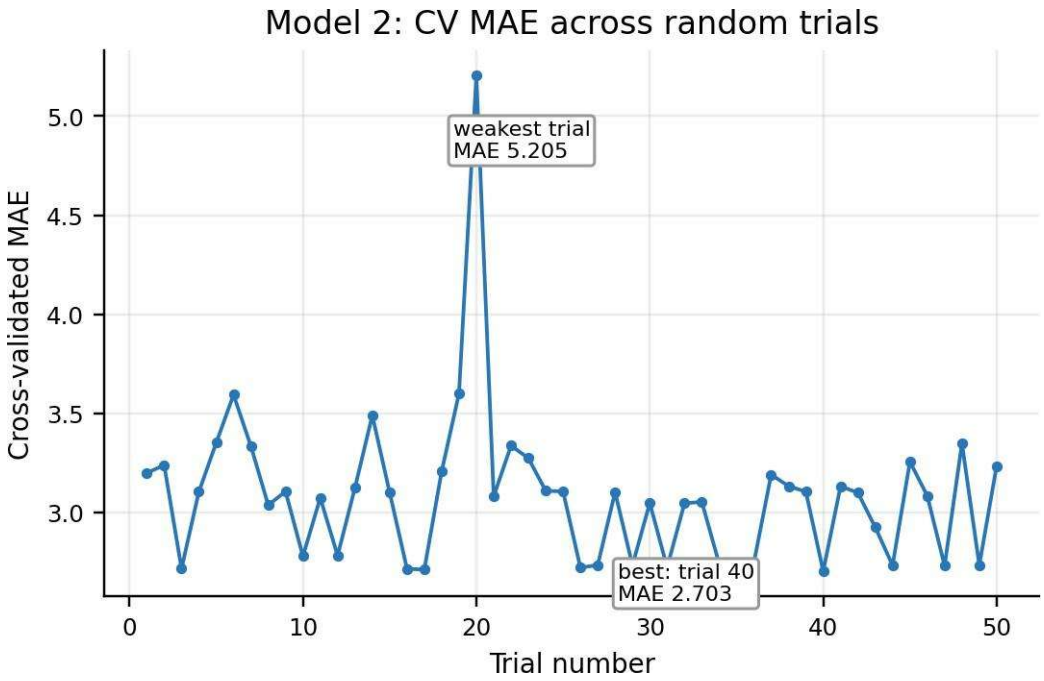


Figure 3. Model 2 trial-by-trial MAE. Lower points are better. The random search explores both strong and weak configurations.

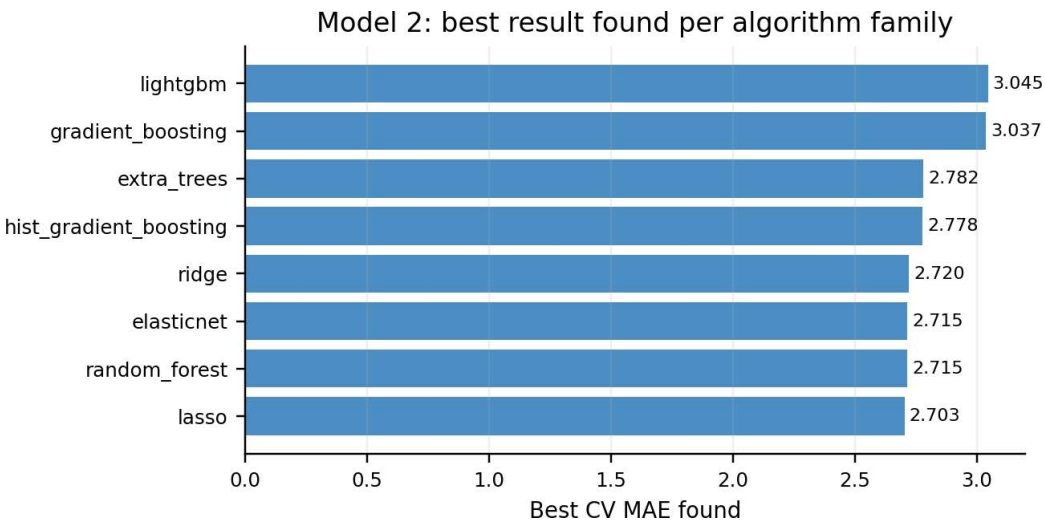


Figure 4. Best Model 2 result found within each algorithm family. The best region is dominated by regularized linear models.

Leaderboard, final fit, and saved submission

After all trials finish, the code sorts the trial results by MAE and prints the top rows. Then it fits the best pipeline on the full training set and predicts wins for the unseen prediction rows. Finally, it rounds the predictions, clips them to the legal baseball

season range, saves them to prediction_submission_2.csv, and prints the first few rows.

Output cell meaning

The printed trial log is a running scoreboard. Each line tells you which algorithm was tested and how large the average prediction error was. For example, **Trial 40** is a Lasso model with **CV MAE = 2.7030**, meaning the model missed by about **2.7 wins** on average across validation folds.

Code Cell 6 - Model 3 Configuration

This cell sets the rules for the ensemble strategy. Compared with Model 2, it adds two important ideas: a **top pool** of good models and an **ensemble size.** An ensemble combines several models instead of trusting only one.

Setting	Value in notebook	Meaning
MODEL3_TOTAL_TRIALS	50	Evaluate 50 randomly configured candidate models.
MODEL3_TOP_POOL_SIZE	10	After ranking the 50 models, keep the best 10 in a shortlist.
MODEL3_ENSEMBLE_SIZE	3	Randomly select 3 models from the shortlist to form the ensemble.
MODEL3_CV_SPLITS	10	Use 10 validation folds for scoring.
MODEL3_USE_GROUP_CV	True	Use GroupKFold so season groups are kept together.
MODEL3_RANDOM_SEED	123	Controls reproducible random sampling for this section.
MODEL3_OUTPUT_FILE	prediction_submission_3.csv	Name of the final ensemble submission file.

Code Cell 7 - Model 3 Execution

This cell performs a second random search, but instead of immediately stopping at the best single model, it stores many strong candidates so an ensemble can be built afterward.

Sampling and scoring candidate pipelines

The function sample_model3_estimator(rng) works much like the earlier sampler, but with slightly different hyperparameter ranges. The cell again calls prepare_datasets() and builds the shared preprocessor. Then, for each of 50 trials, it creates a pipeline and evaluates it using cross-validated MAE.

This time, because MODEL3_USE_GROUP_CV is True, the validation uses **GroupKFold** with season groups. That means all rows from the same year stay together inside the same fold. This is a more era-aware evaluation setup.

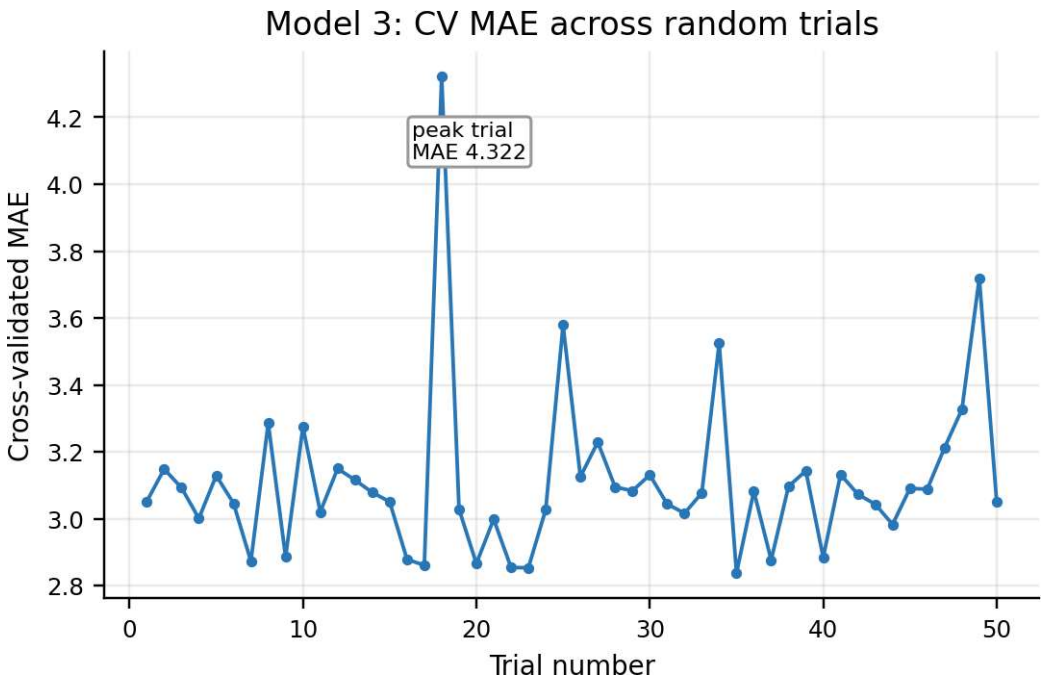


Figure 5. Model 3 trial-by-trial MAE under GroupKFold evaluation.

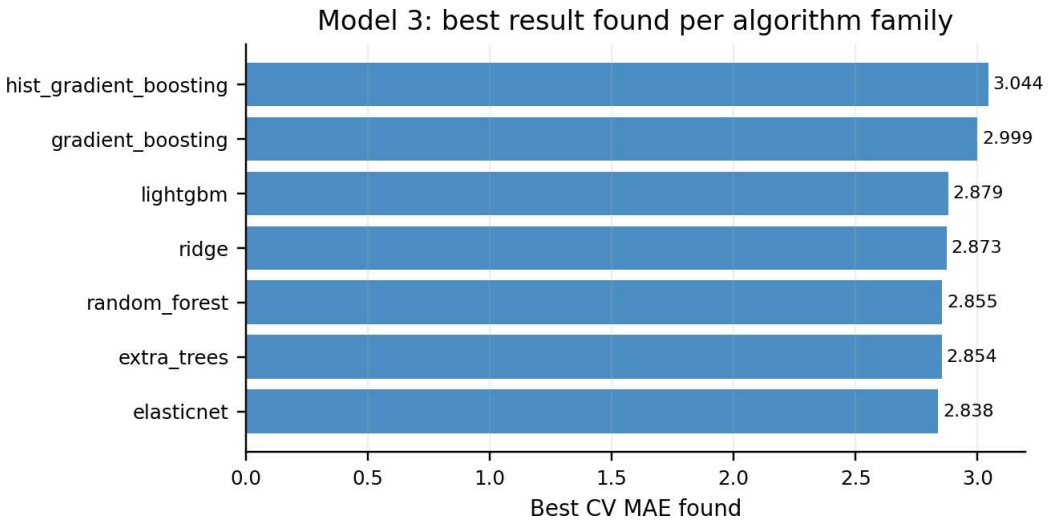


Figure 6. Best Model 3 result found within each algorithm family. Ridge and ElasticNet are strongest here as well.

Code Cell 9 - Output File Check

This last code cell is a simple confirmation step. It creates a small list of the two expected output filenames and checks whether each file exists in the project folder.

The output says both files exist. In plain language, that means the notebook successfully wrote both submission files and the workflow reached its final goal.

Key Learning Points for a New Python and Machine Learning Learner

This notebook teaches several core habits used in real Data Science work:

1. **Build reusable helper functions.**
2. **Create domain-aware features.**
3. **Use preprocessing inside a pipeline.**
4. **Judge models by validation error, not by intuition.**
5. **Separate configuration from execution.**
6. **Export results in a controlled way.**

For someone moving from a non-IT profession into Data Science, a good mental model is this: coding is not just about writing instructions. It is about building a repeatable decision process. This notebook is a repeatable decision process for predicting baseball team wins.

Appendix A - Short Glossary

Term	Simple meaning	Real-life metaphor
Feature engineering	Creating better input columns from raw data.	Turning basic groceries into prepared ingredients before cooking.
Pipeline	A fixed sequence of steps that data goes through.	A car wash tunnel where each car receives the same sequence of cleaning steps.
Cross-validation	Testing a model across several train/validation splits.	Practising a presentation in several rooms to check whether it works everywhere.
MAE	Average size of prediction mistakes. Lower is better.	If your travel-time guesses are off by 5 minutes on average, your MAE is 5 minutes.
Hyperparameter	A model setting chosen before training.	Choosing the oven temperature before baking.
Regularization	A penalty that discourages a model from becoming overly extreme.	A speed governor that stops a vehicle from racing out of control.
Ensemble	Combining several models into one prediction.	Asking several experts and averaging their advice.
Data leakage	Letting answer-related information sneak into training inputs.	Seeing the exam answer sheet before taking the test.

Appendix B - Source Note

Primary source analyzed for this report: moneyball_analytics_automl_ensemble.ipynb (user-provided notebook file).

All code explanations and output interpretations in this report were derived from the notebook's Python cells and the visible outputs stored in that notebook.